
ANDROID SECURITY
Interface Anti-Patterns
Exploiting Insecure Navigation
in Third-Party Android App Lockers

Agenda

Whoami
Android Security Primer
Related Work
Exploit chain
Fixes + hardening patterns
Demos
Conclusion

Primer
Exploit
Mitigations
Demo

WHOAMI

delta.cyberm.ca/research

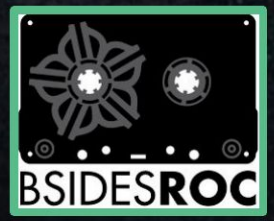
- Principal Security Researcher
- Senior Cybersecurity Analyst @ F500

ACTUATOR SECURITY LABS

WWW.ACTUATOR.SH

SHMOOCON

LESS MOOSE THAN EVER - JAN 10-12, 2025



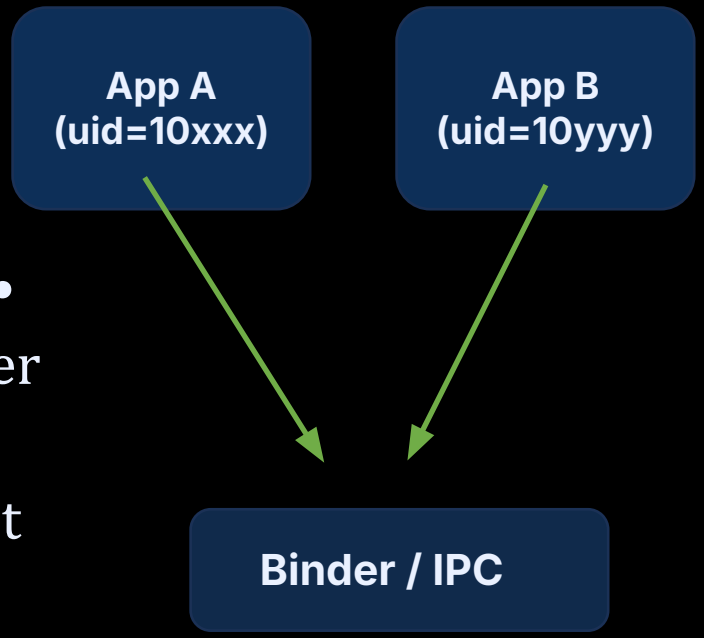
Android Security Crash Course

This class of bug lives at IPC + UI boundaries

Apps are sandboxed
(UID-per-app),
but still communicate via IPC

Key components: Activity • Service • BroadcastReceiver • ContentProvider

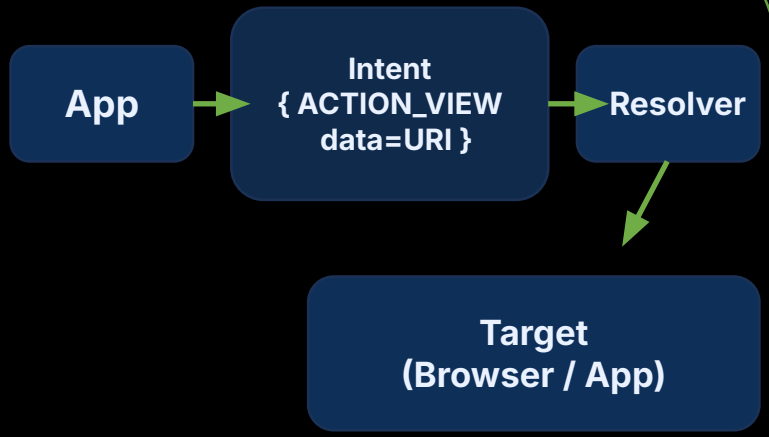
- Most high-impact issues happen at boundaries: intents, exported components, WebViews, overlays



Intents 101: ACTION_VIEW

Implicit routing + attacker-controlled data = unexpected behavior

- Intent = action + data (URI) + extras
- Explicit intent → specific component
- Implicit intent → system resolves



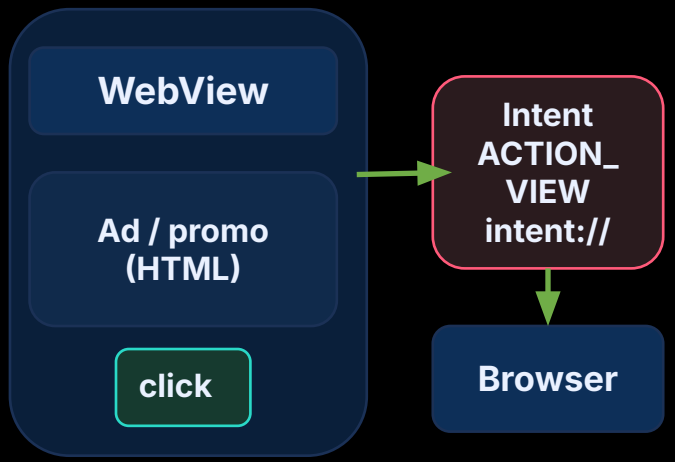
Attacker controls URI payload

Validate scheme/host
Prefer allowlists

WebViews: why ads inside “locked” UI are risky

Trusted click → untrusted route

- WebViews often navigate externally via intents
- Promo/ad inside a lock screen creates a trusted-click → untrusted-route



Lock UI must prevent egress

Related Work

Researchers discovered a lock screen bypass bug in Android 14 and 13 that could expose sensitive data in users' Google accounts.

The security researcher Jose Rodriguez (@VBarraquito) **discovered a new lock screen bypass vulnerability for Android 14 and 13**. A threat actor with physical access to a device can access photos, contacts, browsing history and more.

https://youtu.be/SKefEUqIxrg?si=4rRh0Cy3vMdodw6s

https://securityaffairs.com/155588/hacking/android-14-13-lock-screen-bypass.html

THREAT MODEL

Insecure Navigation Through Exposed Routes Facilitates App Control Evasion [INTERFACE]

- Local user with physical access to unlocked device session
- Target app is configured as “locked” in App Locker
- Ad / promo / WebView element present on lock overlay leads to insecure navigation

Monetization Model

Ads on the Lockscreen

Lockscreen apps maximize ad exposure at the moment of user attention.

High Visibility

Ads are shown upfront on the lockscreen when the user is most likely to see them.

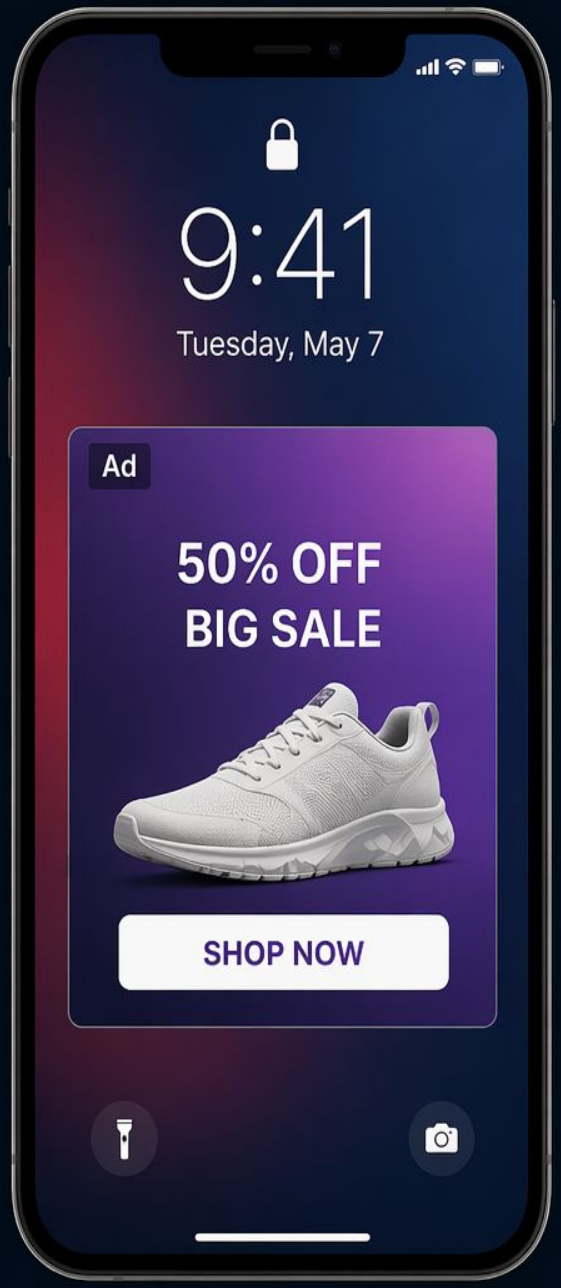
High Volume

Every unlock attempt = another ad impression opportunity.

Revenue Focused

More ads, more impressions, more revenue.

User experience & security take a backseat to ad revenue.



Navigation Flow — Ad ⟶ Browser ⟶ Return

1. User taps protected app — lock overlay appears
2. Ad / promo element inside overlay is tapped
3. WebView launches external browser via ACTION_VIEW / intent://
4. User interacts in browser
5. User returns to protected app

Browser Interaction Outcomes

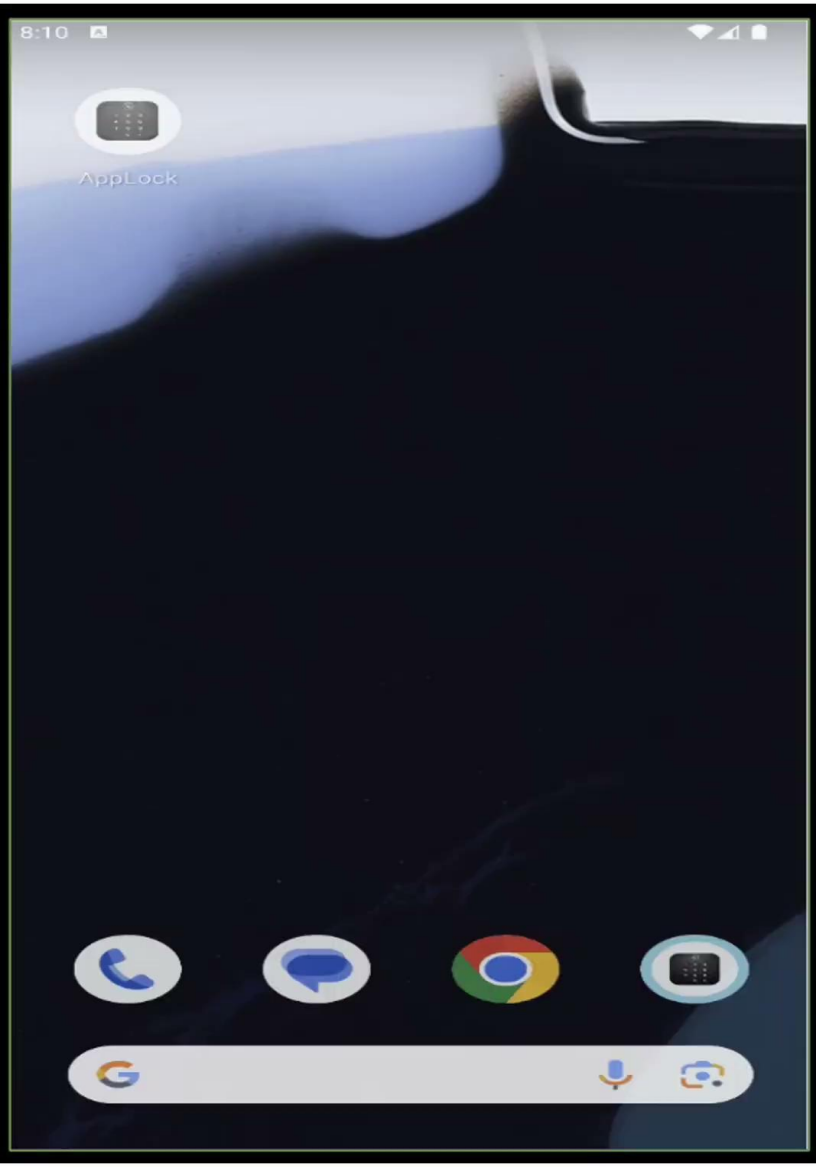
Path A — "Bypass" Case (Timing Met)
- User taps ad → Chrome opens → quickly types/selects URL
- Navigation begins BEFORE delayed overlay executes

Result:
- Overlay does NOT re-attach to Chrome
- Browser remains unlocked AFTER navigation

Path B — Normal Case (Timing Missed)
- Navigation starts too late

Result:
- postDelayed() executes
- Overlay re-attaches
- Browser remains locked

DEMO



Code vs Runtime

Observed overlay pattern:

- show() → overlay added after ~500 ms (postDelayed)
- hide() → may delay removal to satisfy min-visible-time

Security behavior:

- Protected app becomes interactive on return
- Overlay not yet attached
- Gap ≥ 500 ms under common navigation paths
- Result: user interaction window exists BEFORE enforcement.

Code vs Runtime

Observed overlay pattern:

- show() → over
- hide() → may
  min-visible-ti

```
public void hide() {
    this.mDismissed = true;
    removeCallbacks(this.mDelayedShow);
    long currentTimeMillis = System.currentTimeMillis();
    long j10 = this.mStartTime;
    long j11 = currentTimeMillis - j10;
    if (j11 >= 500 || j10 == -1) {
        setVisibility(8);
    } else if (this.mPostedHide) {
    } else {
        postDelayed(this.mDelayedHide, 500 - j11);
        this.mPostedHide = true;
```

Security behavio

- Protected app
- Overlay not ye
- Gap ≥ 500 ms under common navigation paths
- Result: user interaction window exists BEFORE enforcement.

Return Timing & T_GAP

Protected app active → overlay ATTACHED

2 ACTION_VIEW / ad / deep link → startActivity(...) (egress)

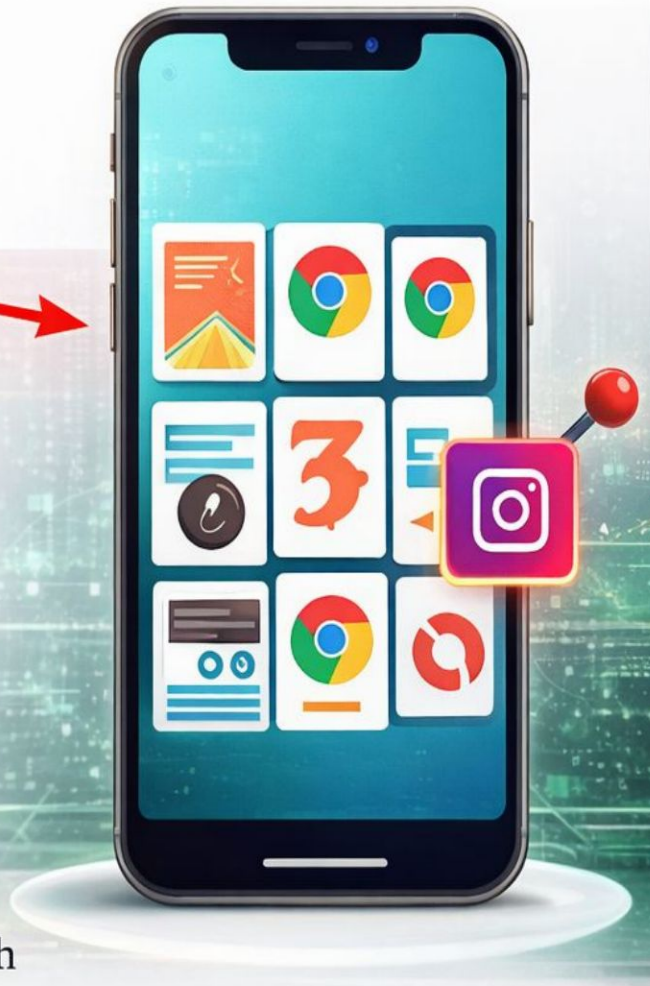
3 **Overlay** callbacks CANCELLED on focus loss

4 User returns (Back / Overview)

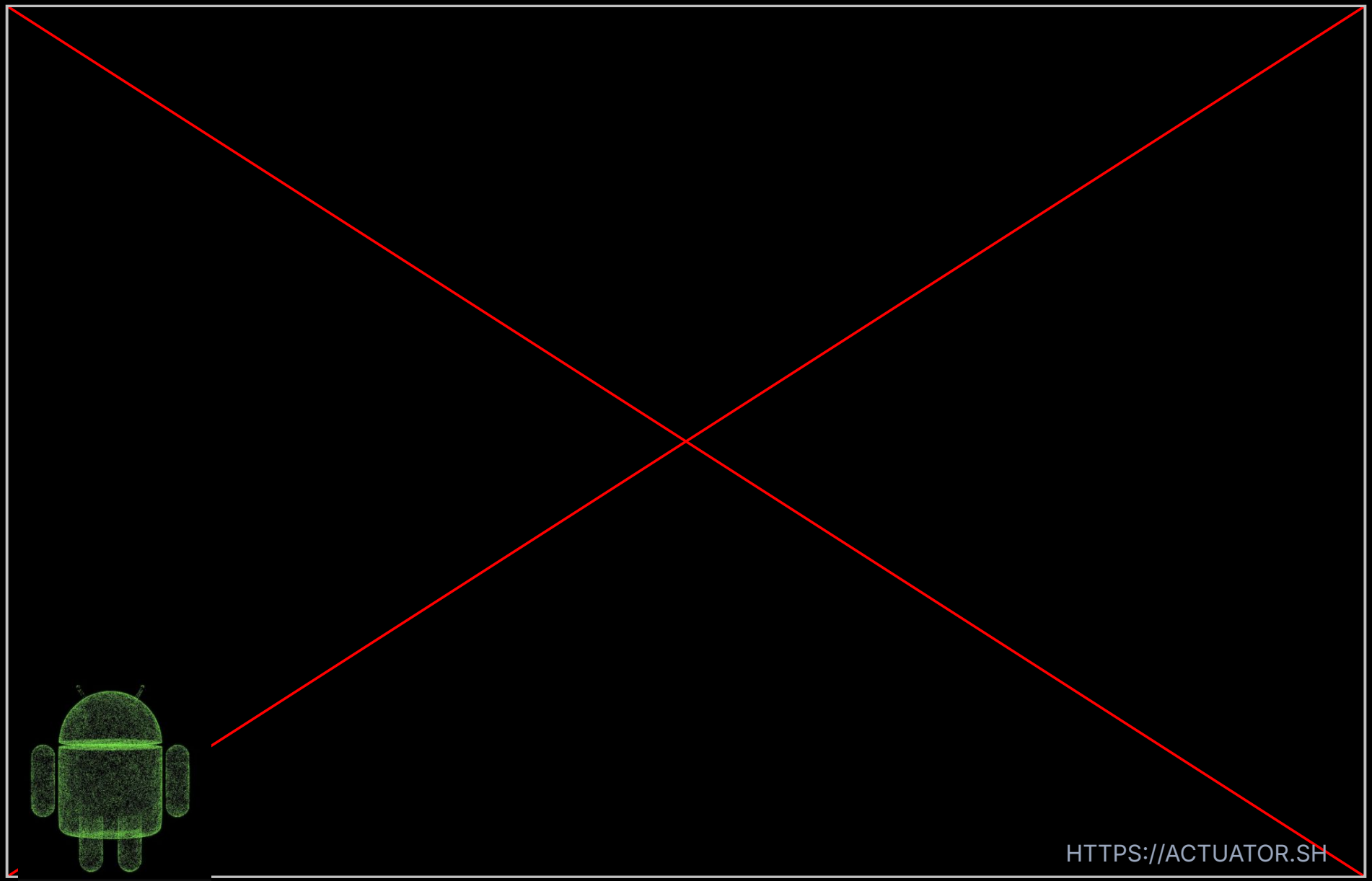
5 Activity becomes INTERACTIVE → t_gap begins

Overlay re-attaches via postDelayed(...) after policy delay

↳ Exploit window = time between INTERACTIVE and re-attach



DEMO



INTERFACE — CASE STUDY

REAL-WORLD ANDROID APPLICATION VULNERABILITIES

	CVE ID
AppLock - Lock apps & Pin lock SailingLab 100M+ Downloads	CVE-2025-68708
AppLock - Fingerprint SpSoft 50M+ Downloads	CVE-2025-68710
App Lock - Fingerprint, Applock Easylife 10M+ Downloads	CVE-2025-68711
App Lock and Fingerprint Lock ApplLockZ - TrustedApp 1M+ Downloads	CVE-2025-68712

CWE-288 | Overlay Authentication Bypass | Insecure Navigation via Exposed Routes

Best (app lock) app?

QUESTION

Features i want in priority :

1- can make the app invincible to (uninstall)

2- safe app

3- free/ 1 time purchase

4- open source

Archived post. New comments cannot be posted and votes cannot be cast.

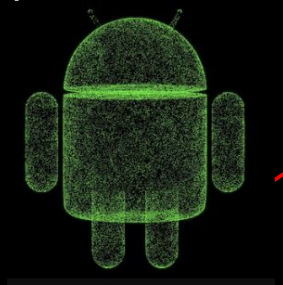
Xisrr1 • 2y ago

Every app lock downloaded can be bypassed. Only app lock that came with the system will do it.

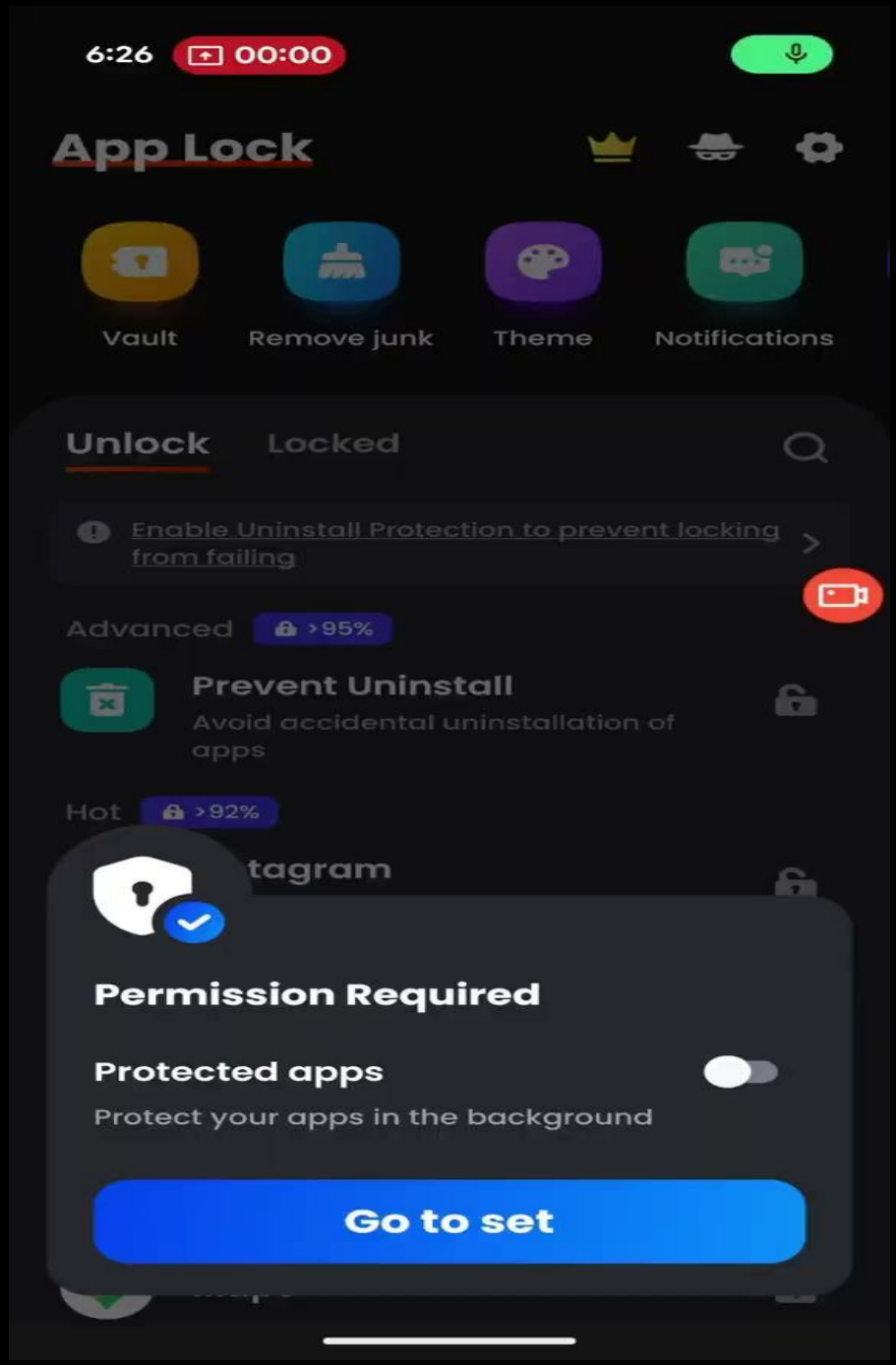
Embarrassed_Habit414 • 2y ago

App lock came a long way hard to uninstall these days. Also I don't think someone will uninstall it, because first of all he will have a phone lock already, but he want a lock app so that his friend and family doesn't snoop around most probably, and if he hands his phone to a friend and he later find the app lock no longer exist then he'll know who did it. App lock can lock itself too, and the settings can always be locked, using a pc to uninstall the lock app well that's not really gonna happen unless he has like something that he is really trying to hide.

DEMO



DEMO



HTTPS://ACTUATOR.SH

Android 15: Can a Play Store App Block Its Own Uninstall?

*Short answer: **No**, not on a personal device.*

A normal third-party app with Device Admin permission cannot permanently prevent the user from uninstalling it on Android 15.

1 What Device Admin (Legacy) Can Do

If the user activates the app as a Device Admin (legacy DeviceAdminReceiver), the app can prevent uninstall while it is active.

What the user must do:

Open Settings (Security / Device admin apps) → **Deactivate / Unregister admin** (Turn it off) → **Uninstall** as usual

So, Device Admin adds an extra step, but does not make the app truly undeletable.

2 What Can Truly Block Uninstall

The stronger API `setUninstallBlocked()` is reserved for enterprise management:

- Device Owner (DO)
- Profile Owner (PO) / Delegated Admin with proper authority

Examples

- Corporate fully managed device
- Work profile managed by company
- MDM / EMM policy control

This is not available to a regular Play Store app that only has user-granted Device Admin.

Key Takeaway: On Android 15, a normal Play Store app with Device Admin can delay uninstall by requiring deactivation first, but it cannot permanently prevent the user from removing the app on their own device.

Reference: Android Developers Documentation | *"To uninstall an existing device admin app, users need to first unregister the app as an administrator."* developer.android.com/work/device-admin

Top 3 Mitigations

- Consider removing advertisements from lock screen
- Block intent:// navigation while locked
- Remove timing gaps – re-gate synchronously onResume() / fail-closed

But what about End Users?

~~TOP 3 MITIGATIONS~~

UPGRADE TO ANDROID 17

Private space

Private Space enables users to create a secure, isolated environment on their device to keep sensitive apps away from prying eyes. Apps in the private space show up in a separate container in the launcher, and are hidden from the recents view, notifications, settings, and from other apps when the private space is locked.

The sandboxed space is a separate Android profile When the end user adds or installs an app inside private space, the app is installed in this new Android profile. The system treats this as a fresh app install, and no app data is copied over to the private space. When the space is locked, the private profile user is stopped, and when the space is unlocked, the user is started.

Apps in the private space are installed as separate copies of the apps in the main space. User content (user-generated or downloaded) and user accounts are separated between the private space and the main space. You can use the system Sharesheet and the Photo Picker to give apps access to content across spaces only when the private space is unlocked.

Private space is based on the Android multi-user model and adds the following profile and usertype:

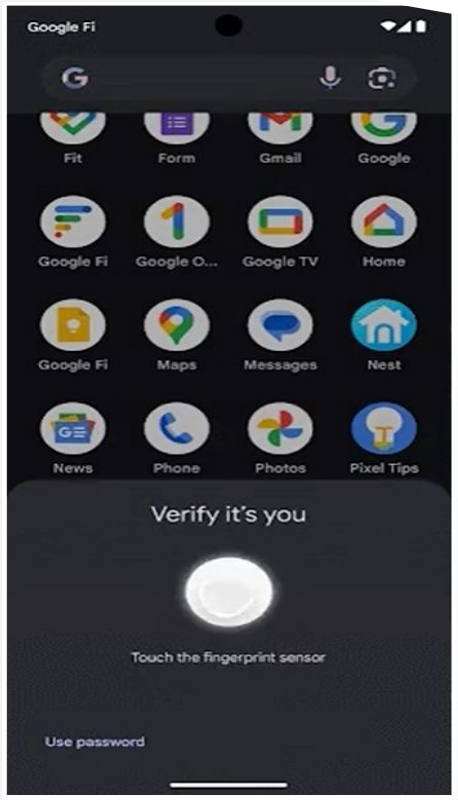
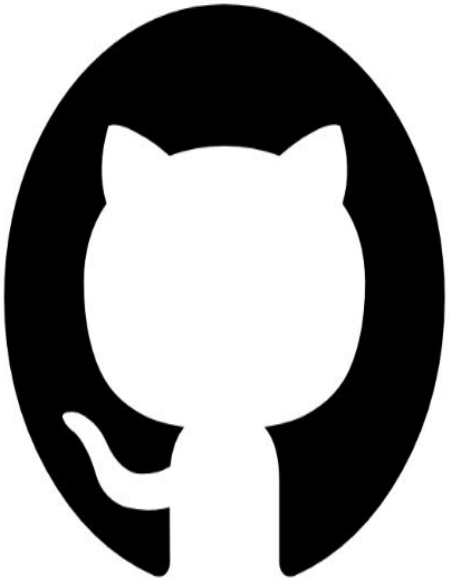


Figure1. *The private space can be unlocked and locked to show or hide sensitive apps on a device.*

https://source.android.com/docs/security/features/private-space



github.com/actuator/ekoparty

THANK YOU!

SOCIALS:

HTTPS://GITHUB.COM/ACTUATOR/EKOPARTY

HTTPS://WWW.YOUTUBE.COM/@ACTUATOR

HTTPS://INFOSEC.EXCHANGE/@ACTUATOR